量化交易进阶:第49天 – LightGBM & CatBoost 用梯度提升模型预测收益与风险

原创 逻辑与流 逻辑与流 2025年11月12日 11:19 北京 听全文

主题：使用机器学习模型（树模型）从多因子特征中预测未来收益或风险。

一、学习目标

今天我们要解决两个关键问题：

1. 如何用 LightGBM / CatBoost 训练模型预测未来收益?
2. 如何从模型中提取特征重要性（Feature Importance），指导因子筛选?
3. 如何评估模型效果：IC、分层收益、AUC、回测稳定性?

二、树模型在量化中的优势

1. 梯度提升树（GBDT）思想

梯度提升模型是一种**集成学习（Ensemble Learning）**方法：

用多个弱模型（决策树）叠加形成一个强预测器。

模型公式：

$$\hat{y} = \sum_{m=1}^{M} \gamma_m \, f_m(x)$$

其中每个 $f_m(x)$是一棵决策树，

模型不断拟合前一轮的残差（gradient），逐步逼近真实收益。

1.1 公式的整体含义

$$\hat{y} = \sum_{m=1}^{M} \gamma_m \, f_m(x)$$

这是梯度提升模型（GBM, LightGBM, XGBoost, CatBoost等）的预测表达式， 表示模型是**多个弱学习器（通常是决策树）**的加权和。

- $\hat{y}$：最终预测值（模型输出）
- $f_m(x)$：第 m 棵树的输出（弱学习器）
- γ_m：该树的权重（步长 / 学习率 / 最优系数）
- M：总共迭代的树的数量

换句话说，模型在训练中逐步叠加一棵棵树来修正残差， 每棵树都试图学习前面所有树预测错误的部分。

1.2 系数 γ_m的意义

每一项的系数 γ_m代表了 这一轮弱学习器在整体模型中的贡献大小。

梯度提升训练的过程是这样的： 1. 初始化模型

$$\hat{y}^{(0)} = \arg\min_{\gamma} \sum_i L(y_i, \gamma)$$

比如在回归问题中，可能直接取均值。

2． 每一轮迭代
- 计算上一轮预测的残差（或者梯度方向）：

$$r_i^{(m)} = -\frac{\partial L(y_i, \hat{y}_i^{(m-1)})}{\partial \hat{y}_i^{(m-1)}}$$

- 拟合一个新树 `$f_m(x)$` 来逼近残差。
- 然后找到最佳的 `$\gamma_m$`（该树的整体缩放系数）：

$$\gamma_m = \arg\min_{\gamma} \sum_i L\Big(y_i, \hat{y}_i^{(m-1)} + \gamma f_m(x_i)\Big)$$

- 更新预测：

$$\hat{y}^{(m)} = \hat{y}^{(m-1)} + \nu \gamma_m f_m(x)$$

其中 $\nu \in (0, 1]$是全局学习率（shrinkage）。

1.3 直观理解

- 每一轮加的 $f_m(x)$是一个“小修正”；
- γ_m告诉模型：这一棵树到底要修正多少；
- ν是全局的“油门控制”，避免一次修得太多导致过拟合；
- 所以模型其实是一个：

$$\text{baseline} + \text{若干小步修正的和}$$

1.4 类比帮助理解

如果你在预测收益率 $\hat{y}$，那么可以理解为：

- 第 1 棵树预测了一个大致方向（整体趋势）；
- 第 2 棵树修正了其中一些误差；
- 第 3 棵树修正了更细的残差；

- 每棵树修正的强度由 γ_m决定。

最终：

$$\hat{y} = \text{趋势} + \text{修正} + \text{修正} + \dots$$

1.5 举个简单例子

假设目标是回归，损失函数是 MSE：

$$L(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$$

则梯度：

$$r_i^{(m)} = y_i - \hat{y}_i^{(m-1)}$$

最优步长：

$$\gamma_m = 1 \quad (\text{对于MSE是固定的})$$

所以模型变成简单的：

$$\hat{y} = \sum_{m=1}^{M} \nu f_m(x)$$

这就是 LightGBM/XGBoost 回归树 最基本的原理。

符号 含义 数学作用

$f_m(x)$第 m 棵树 学习残差方向的修正

γ_m权重 / 步长 控制树的修正幅度

ν学习率 全局缩放系数，防止过拟合

M树的数量 控制模型复杂度

下面是一个完整可运行的 Python 代码，用来绘制 梯度提升（Gradient Boosting）逐步修正预测过程图，并展示每一轮拟合残差和叠加预测的变化曲线。

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeRegressor

plt.rcParams['font.sans-serif'] = ['PingFang HK']   # 或者 'Songti SC'、'STHeiti'

plt.rcParams['axes.unicode_minus'] = False
```

```python
# ======================
# 1️⃣ 生成模拟数据
# ======================

np.random.seed(42)
X = np.linspace(0, 10, 100).reshape(-1, 1)
y_true = np.sin(X).ravel()

y = y_true + np.random.normal(0, 0.1, size=X.shape[0])  # 加入噪声

# ======================
# 2️⃣ 初始化
# ======================

n_estimators = 5        # 拟合轮数
learning_rate = 0.3     # 学习率
models = []
y_pred = np.zeros_like(y)

# ======================
# 3️⃣ 逐步拟合残差
# ======================

plt.figure(figsize=(10, 6))
plt.plot(X, y_true, 'k--', label='真实函数 (sin(x))')
plt.scatter(X, y, s=15, color='black', alpha=0.6, label='带噪声数据')

for m in range(n_estimators):
    # 计算残差
    residual = y - y_pred
    # 拟合残差
    tree = DecisionTreeRegressor(max_depth=3, random_state=m)
    tree.fit(X, residual)

    # 更新预测
    update = learning_rate * tree.predict(X)
    y_pred += update
    models.append(tree)

    # 画出每一轮的预测曲线
    plt.plot(X, y_pred, label=f'第 {m+1} 轮累积预测')

plt.title("梯度提升逐步修正过程", fontproperties="SimHei", fontsize=14)
plt.xlabel("X", fontproperties="SimHei")
plt.ylabel("预测值", fontproperties="SimHei")
plt.legend(prop={"family": "SimHei", "size": 9})

plt.grid(True)
plt.show()
```

图像说明：

运行后你会看到：

- 黑色虚线：真实函数 $y = \sin(x)$
- 黑点：含噪声的数据（模型训练目标）
- 彩色曲线：每一轮的累计预测结果（第1~5轮）

每一轮曲线都更接近真实函数，展示了残差逐步修正 → 模型累积提升的过程。

总结

为什么 GBDT 适合因子建模

特点 解释

自动处理非线性 不需要人工构造高阶项

变量筛选能力强 可自动识别重要因子

抗异常值能力强 不依赖均值方差假设

可解释性强 可输出 feature importance

三、LightGBM 实战：预测未来 5 日收益

我们用 Tushare 获取真实数据，选取一批股票样本进行训练。

示例目标：预测股票未来 5 日收益率是否为正（分类问题）。

```
import numpy as np
import pandas as pd
import tushare as ts
import lightgbm as lgb
```

```python
import matplotlib.pyplot as plt

from datetime import datetime
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score, classification_report

# ==============================
# 1️⃣ 初始化与参数
# ==============================

token = ""
pro = ts.pro_api(token)

start_date = "20230101"
end_date = "20241231"

merge_start = pd.to_datetime(start_date)
merge_end = pd.to_datetime(end_date)

# ==============================
# 2️⃣ 获取沪深300股票池
# ==============================
hs300 = pro.index_weight(index_code="000300.SH", trade_date=end_date)

if hs300.empty:
    fallback_date = "20240131"
    hs300 = pro.index_weight(index_code="000300.SH", trade_date=fallback_date)

if hs300.empty:
    raise RuntimeError("无法获取沪深300成分股，请检查 Token 权限或 trade_date 配置。")

symbols = hs300["con_code"].unique()[:30]  # 仅取前30只演示

# ==============================
# 3️⃣ 拉取行情与因子数据
# ==============================

dfs = []
roe_info = []

for code in symbols:
    df_basic = pro.daily_basic(
        ts_code=code,
        start_date=start_date,
        end_date=end_date,
        fields="ts_code,trade_date,pe,pb,ps"
    )

    df_price = pro.daily(
        ts_code=code,
        start_date=start_date,
        end_date=end_date,
        fields="ts_code,trade_date,close"
```

```
    )

    if df_basic.empty or df_price.empty:
        roe_info.append({"ts_code": code, "roe_non_null_obs": 0})
        continue

    df = pd.merge(df_basic, df_price, on=["ts_code", "trade_date"], how="inner")
    if df.empty:
        roe_info.append({"ts_code": code, "roe_non_null_obs": 0})
        continue

    df["trade_date"] = pd.to_datetime(df["trade_date"])
    df = df.sort_values("trade_date")
    df = df[(df["trade_date"] >= merge_start) & (df["trade_date"] <= merge_end)]

    # 计算未来5日收益与标签
    df["close_next5"] = df["close"].shift(-5)
    df["future_ret"] = df["close_next5"] / df["close"] - 1
    df["label"] = (df["future_ret"] > 0).astype(int)
    df = df.drop(columns=["close_next5"])

    # 拉取 fina_indicator 获取 ROE，并按公告日向后填充
    df_fina = pro.fina_indicator(
        ts_code=code,
        start_date=start_date,
        end_date=end_date,
        fields="ts_code,ann_date,roe"
    )

    if not df_fina.empty:
        df_fina = df_fina.dropna(subset=["roe"])
        if not df_fina.empty:
            df_fina["ann_date"] = pd.to_datetime(df_fina["ann_date"])
            df_fina = df_fina.sort_values("ann_date")
            df = pd.merge_asof(
                df.sort_values("trade_date"),
                df_fina[["ann_date", "roe"]],
                left_on="trade_date",
                right_on="ann_date",
                direction="backward"
            )

            df = df.drop(columns=["ann_date"])

    if "roe" not in df.columns:
        df["roe"] = np.nan

    roe_non_null = int(df["roe"].notna().sum())
    roe_info.append({"ts_code": code, "roe_non_null_obs": roe_non_null})

    df = df.dropna(subset=["pe", "pb", "ps", "future_ret"])
```

```
    if df.empty:
        continue

    dfs.append(df[["trade_date", "ts_code", "pe", "pb", "ps", "roe", "future_ret", "label"]])

if not dfs:
    raise RuntimeError("未获取到有效的行情/因子数据，无法建模。")

data = pd.concat(dfs, ignore_index=True)
feature_cols = ["pe", "pb", "ps"]
if data["roe"].notna().any():
    feature_cols.append("roe")

print("样本量：", len(data))
print("使用因子：", feature_cols)
print("\nROE 可用性统计：\n", pd.DataFrame(roe_info))

# ==============================
# 4️⃣ 因子预处理
# ==============================

X = data[feature_cols].copy()
y = data["label"]

X = X.replace([np.inf, -np.inf], np.nan).fillna(X.median())

for col in X.columns:
    lower, upper = X[col].quantile(0.01), X[col].quantile(0.99)
    X[col] = X[col].clip(lower, upper)

# ==============================
# 5️⃣ 训练 LightGBM 模型
# ==============================

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

train_data = lgb.Dataset(X_train, label=y_train)

test_data = lgb.Dataset(X_test, label=y_test)

params = {
    "objective": "binary",
    "metric": "auc",
    "learning_rate": 0.05,
    "num_leaves": 15,
    "max_depth": 5,
    "feature_fraction": 0.8,
    "bagging_fraction": 0.8,
    "bagging_freq": 5,
    "verbose": -1
}
```

```
model = lgb.train(
    params,
    train_data,
    valid_sets=[test_data],
    num_boost_round=200,
    callbacks=[lgb.early_stopping(20), lgb.log_evaluation(20)]
)

# ==============================
# 6️⃣ 模型评估
# ==============================

y_pred = model.predict(X_test)
auc = roc_auc_score(y_test, y_pred)

print(f"\n📊 测试集AUC: {auc:.3f}")

print(classification_report(y_test, y_pred > 0.5))

# ==============================
# 7️⃣ 特征重要性
# ==============================

importance = pd.DataFrame({
    "feature": feature_cols,
    "importance": model.feature_importance()
}).sort_values("importance", ascending=False)

print("\n特征重要性：\n", importance)

plt.rcParams["font.sans-serif"] = ["Heiti TC"]
plt.figure(figsize=(6, 4))
plt.barh(importance["feature"], importance["importance"])

plt.title("LightGBM 特征重要性")
plt.xlabel("Importance")
plt.gca().invert_yaxis()
plt.grid(True, linestyle="--", alpha=0.6)

plt.tight_layout()
plt.show()
```

输出：

类别	precision	recall	f1-score	support
0	0.60	0.61	0.60	2072
1	0.59	0.59	0.59	2023
accuracy	0.60	0.60	0.60	4095
macro avg	0.60	0.60	0.60	4095
weighted avg	0.60	0.60	0.60	4095

特征重要性：

feature	importance
pb	770
pe	765
ps	745
roe	501

- 测试集AUC: 0.642：模型在区分未来 5 日上涨 (1) 与下跌 (0) 时，比随机水平（0.5）好一些，但仍有提升空间。
- 分类报告：
- 0 类（未来下跌）：precision=0.60 表示模型把一只股票预测为下跌时有 60% 准确；recall=0.61 表示实际下跌的股票中有 61% 被识别出来；f1-score=0.60 是二者的折中。
- 1 类（未来上涨）：precision=0.59、recall=0.59、f1-score=0.59，说明模型对上涨的识别力度和准确度与下跌相近。
- support 是各类别在测试集中的样本数（下跌 2072、上涨 2023），基本均衡。

- accuracy=0.60：总预测正确率 60% 左右；宏平均和加权平均的 precision/recall/f1 也都在 0.60，说明两个类别表现接近，没有明显偏向。

这是 LightGBM 给出的特征重要性统计，数值越大表示模型越常用该因子来划分样本、对预测贡献越大：

- pb（市净率）得分 770，当前样本中它最有助于判断未来 5 日涨跌。
- pe（市盈率）紧随其后，说明估值高低也在模型里体现了作用。
- ps（市销率）与前两项相近，也提供较多信息。
- roe（净资产收益率）虽然得分略低（501），但仍然被模型频繁使用，说明通过 fina_indicator 合并后的 ROE 确实参与了预测。若想增强其影响，可以尝试拓展时间窗口、做季节性平滑或加入同比/环比变化等派生指标。

四、模型解读

- AUC > 0.6：模型具备一定区分能力；
- 特征重要性（Feature Importance） 能告诉我们哪些因子在预测未来收益中贡献最大；
- 若"ROE"或"PE"重要性高，说明市场近期更关注盈利能力因子。

AUC 是机器学习和量化投资中最常用的模型评估指标之一，全称是：

$$AUC = \text{Area Under the ROC Curve}$$

即 "ROC 曲线下的面积"。

它主要用于衡量二分类模型的整体区分能力，比如：

- 模型能否把"涨的股票"和"跌的股票"区分开；
- 模型能否把"好样本"和"坏样本"排序正确。

4.1 核心直觉

假设你训练了一个模型，输出每只股票"明天上涨的概率"：

股票 实际涨跌 模型预测的上涨概率

A 涨 0.9

B 跌 0.2

C 涨 0.8

D 跌 0.3

直觉上：

- 模型希望“涨的股票概率高”，“跌的股票概率低”；
- AUC 衡量的就是这种排序能力。

4.2 数学定义

AUC 是 ROC 曲线下的面积，而 ROC 曲线由以下两维构成：

坐标轴 含义

x轴：FPR（False Positive Rate） 错把跌的股票当成涨的比例

y轴：TPR（True Positive Rate） 正确识别出涨的股票的比例

随着分类阈值从高到低移动，模型会生成不同的 (FPR, TPR) 点，连接起来形成一条 ROC 曲线。

AUC 则是：

$$\text{AUC} = \int_0^1 TPR(FPR)\, d(FPR)$$

数值范围：

- AUC = 1.0 → 完美区分（涨的都排前面）
- AUC = 0.5 → 完全随机（模型没用）
- AUC < 0.5 → 排序反了（涨跌颠倒）

4.3 直观解释

AUC 就是“随机抽取一只涨的股票”和“一只跌的股票”，模型预测得分更高的那只为“涨”的概率。

4.4 Python 实战示例

下面演示如何计算并绘制 ROC 曲线与 AUC：

```
import numpy as np
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt


# 模拟模型预测结果
y_true = np.array([1, 0, 1, 0, 1, 0, 1, 0])   # 实际涨跌（1涨，0跌）
y_score = np.array([0.9, 0.1, 0.8, 0.4, 0.7, 0.3, 0.6, 0.2])  # 模型预测分数

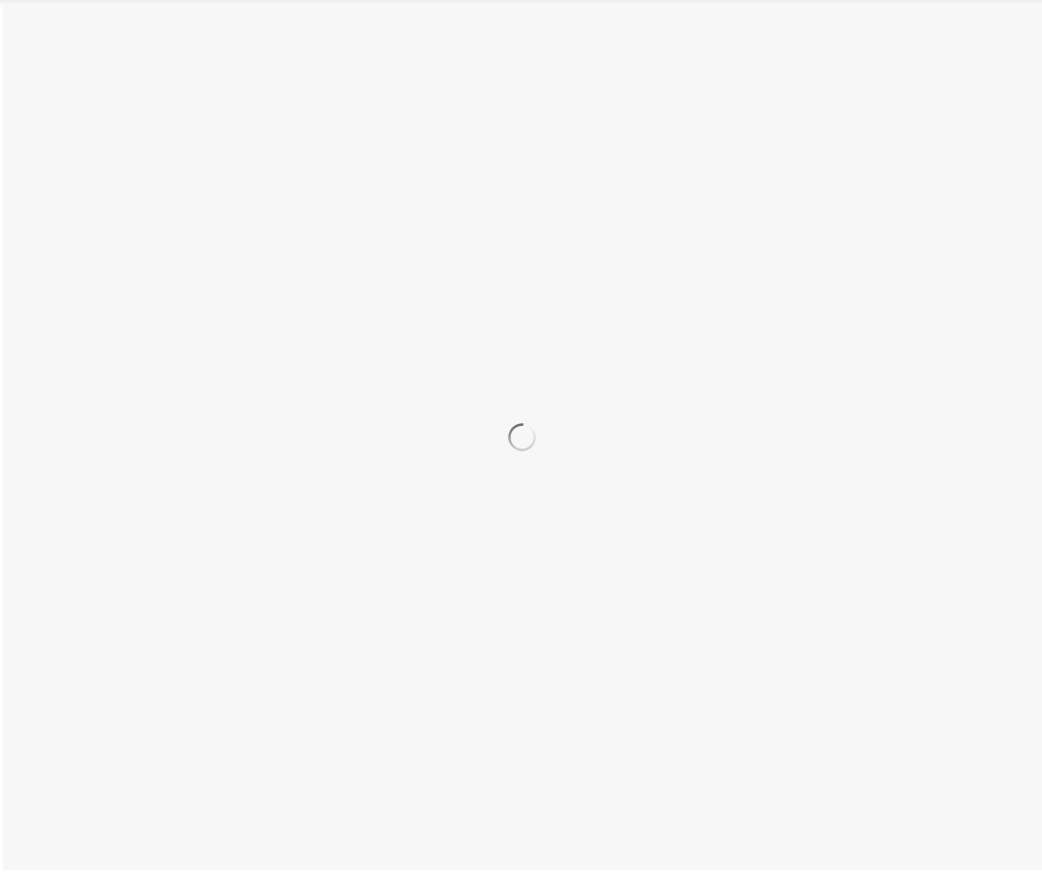
# 计算 ROC 曲线与 AUC
fpr, tpr, _ = roc_curve(y_true, y_score)
roc_auc = auc(fpr, tpr)

# 绘制图像
plt.figure(figsize=(6, 5))
```

```
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC曲线 (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=1, linestyle='--', label='随机分类线')
plt.xlabel('FPR（假正率）', fontproperties='SimHei')
plt.ylabel('TPR（真正率）', fontproperties='SimHei')
plt.title('AUC 指标示意图', fontproperties='SimHei')
plt.legend(prop={"family": "SimHei"})

plt.grid(True)

plt.show()
```



五、量化投资中的应用

在量化中，AUC 可用于：

- 因子预测涨跌方向的准确度衡量；
- 择时模型的优劣比较；
- 分类风险模型（如是否触及止损）的判断。

例如：

若你的收益预测因子能把“未来涨幅高的股票”排在前列，

那么它的 AUC 就会接近 1.0。

总结一句话：

AUC 衡量的是模型区分正负样本（涨/跌）的整体能力。 它不依赖阈值、不受收益尺度影响，非常适合做因子有效性评估。

如果你理解了 LightGBM 或 XGBoost，那么理解 CatBoost 会非常顺畅——它们都属于同一类模型：

梯度提升决策树（Gradient Boosting Decision Tree, GBDT）家族。

不过 CatBoost 是这个家族中**最"聪明"、最自动化"**的一员，尤其擅长处理类别型（categorical）特征。

五、CatBoost：类别特征友好的梯度提升

5.1 CatBoost 是什么?

CatBoost = Categorical + Boosting

由 Yandex（俄罗斯的"百度"） 开发，核心目标是解决传统 GBDT 的两个问题： 1. 类别特征难处理（如"行业"、"省份"、"交易所"等） 2. 训练速度慢、过拟合严重

CatBoost 通过数学优化和工程改进，让模型能：

- 自动识别类别特征；
- 无需人工独热编码（One-Hot）；
- 拟合更稳定；
- 更快、更省内存。

5.2 CatBoost 的核心创新

问题	XGBoost/LightGBM	CatBoost 的改进
类别特征	需手动编码	自动转换为"统计编码"
梯度偏移（梯度泄露）	可能过拟合	使用 Ordered Boosting 解决
参数复杂度	超多参数	几乎开箱即用
训练速度	快	同样快，甚至更稳
特征交互	手动构造	自动特征组合（Interaction Detection）

5.3 核心思想：Ordered Boosting（有序提升）

传统 GBDT 在每一轮训练时，会用当前样本的真实标签来计算梯度， 这容易导致模型"偷看答案"，出现 目标泄漏（Target Leakage）。

CatBoost 的解决方法是：

在训练每棵树时，只用在此样本"之前"出现的样本信息。 这样即便在随机顺序下，也避免了泄漏。

5.4 类别特征处理（Category Encoding）

假设有一个类别特征： 股票代码 行业 000001 银行 000002 地产 000003 银行 000004 科技 CatBoost 会自动转换为"统计目标编码"：

$$\text{编码值} = \frac{\text{该类别的正样本数} + \text{prior}}{\text{该类别的总样本数} + \text{平滑项}}$$

例如“银行”行业可能被转化为“上涨概率 = 0.7”，“科技”=0.4。

这一过程无需人工干预，非常适合股票行业、板块、地区等离散变量。

5.5 Python 实战示例

下面我们用一个简单例子演示 CatBoost 的使用。

```
from catboost import CatBoostClassifier, Pool

from sklearn.model_selection import train_test_split

from sklearn.metrics import roc_auc_score

import pandas as pd

# 模拟数据

data = pd.DataFrame({
    '行业': ['银行', '科技', '银行', '地产', '科技', '银行', '地产', '科技'],
    '市盈率': [8, 35, 10, 15, 40, 12, 18, 30],
    '涨跌': [1, 0, 1, 0, 0, 1, 0, 1]  # 1表示上涨
})

# 拆分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(
    data[['行业', '市盈率']], data['涨跌'], test_size=0.3, random_state=42
)

# 指定类别特征列
cat_features = ['行业']
# 建立 CatBoost 模型
model = CatBoostClassifier(
    iterations=100,
    learning_rate=0.1,
    depth=3,
    loss_function='Logloss',
    verbose=0
)

# 训练
train_pool = Pool(X_train, y_train, cat_features=cat_features)
test_pool = Pool(X_test, y_test, cat_features=cat_features)
model.fit(train_pool)

# 预测
y_pred = model.predict_proba(test_pool)[:, 1]
```

```
auc = roc_auc_score(y_test, y_pred)
print(f"AUC = {auc:.3f}")
```

5.6 量化中的应用

在量化投资里，CatBoost 非常适合：

应用场景	示例
多因子融合	输入财务因子 + 技术因子 + 行业分类
风险模型	预测个股是否会暴跌
择时策略	判断市场短期上行概率
高维特征	自动处理数百个原始因子

它尤其擅长“混合数据类型”问题（数值 + 类别），比如：

用股票的市盈率（数值）+ 行业（类别）+ 板块（类别）预测未来 5 日收益率方向。

5.7 小结

特点	含义
模型类型	梯度提升决策树（GBDT）
开发团队	Yandex
优势	自动处理类别特征、抗过拟合、速度快
关键机制	Ordered Boosting、Target Encoding
在量化中的用法	因子融合、分类择时、风险预测

做一个真正完整的量化实战例子：

用 CatBoost + Tushare 预测未来 5 日涨跌（分类任务）。

5.8 实战思路总览

1. 获取数据：用 tushare 获取沪深 A 股一年的日线数据、财务指标（PE、PB、ROE）。
2. 计算目标变量：
    - target = 1 → 未来5日收益 > 0
    - target = 0 → 否则
3. 特征工程：
    - 填充缺失值
    - Winsorize 去极值
    - 标准化数值特征
    - 行业编码为类别特征（CatBoost天然支持）

4. 训练模型：

- CatBoostClassifier
- 验证集评估 AUC

5. 可视化结果：

- ROC 曲线
- AUC 值

5.9 安装依赖

```
pip install tushare catboost scikit-learn matplotlib pandas scipy
```

5.10 完整实战代码

```
import tushare as ts
import pandas as pd
import numpy as np

from catboost import CatBoostClassifier, Pool
from sklearn.metrics import roc_auc_score, roc_curve
from sklearn.model_selection import train_test_split
from scipy.stats.mstats import winsorize

import matplotlib.pyplot as plt
from matplotlib import font_manager

# 设置 Tushare token
ts.set_token('')
pro = ts.pro_api()

# 1️⃣ 获取一只实际股票的基本面数据，例如贵州茅台
ts_code = '600519.SH'

# 获取过去一年的日线行情

df_price = pro.daily(ts_code=ts_code, start_date='20240101', end_date='20241101')
df_price = df_price.sort_values('trade_date')
df_price['return'] = df_price['close'].pct_change()

# 获取市值相关指标（PE、PB、ROE）
df_val = pro.daily_basic(ts_code=ts_code, start_date='20240101', end_date='20241101',

                         fields='ts_code,trade_date,pe,pb,roe')

df_val = df_val.sort_values('trade_date')

# 合并行情与估值
df = pd.merge(df_price, df_val, on=['ts_code', 'trade_date'], how='inner')
```

```
df['trade_date'] = pd.to_datetime(df['trade_date'])
df = df.sort_values('trade_date')

# 若 daily_basic 未返回 roe，先补列

if 'roe' in df.columns:
    df = df.rename(columns={'roe': 'roe_daily'})
else:
    df['roe_daily'] = np.nan

# 使 fina_indicator 补充 ROE
df_fina = pro.fina_indicator(
    ts_code=ts_code,
    start_date='20230101',
    end_date='20241101',
    fields='ts_code,ann_date,roe'

)

if not df_fina.empty:
    df_fina = df_fina.dropna(subset=['roe'])
    if not df_fina.empty:
        df_fina['ann_date'] = pd.to_datetime(df_fina['ann_date'])
        df_fina = df_fina.sort_values('ann_date')
        df_fina = df_fina.rename(columns={'roe': 'roe_fina'})
        df = pd.merge_asof(
            df,
            df_fina[['ann_date', 'roe_fina']],
            left_on='trade_date',
            right_on='ann_date',
            direction='backward'
        )

        df['roe'] = df['roe_fina'].combine_first(df['roe_daily'])

        df = df.drop(columns=['roe_fina', 'roe_daily', 'ann_date'], errors='ignore')
    else:
        df['roe'] = df['roe_daily']
        df = df.drop(columns=['roe_daily'], errors='ignore')
else:
    df['roe'] = df['roe_daily']
    df = df.drop(columns=['roe_daily'], errors='ignore')

# 若仍缺少列，继续补齐
missing_cols = [col for col in ['pe', 'pb', 'roe'] if col not in df.columns]

for col in missing_cols:
    df[col] = np.nan

df = df.dropna(subset=['pe', 'pb'])
```

```
# 2️⃣ 计算未来5日涨跌作为目标
df['future_return'] = df['close'].shift(-5) / df['close'] - 1
df['target'] = (df['future_return'] > 0).astype(int)

# 3️⃣ 特征工程
features = ['pe', 'pb', 'roe']
# 缺失值填充
df[features] = df[features].fillna(df[features].median())

# Winsorize 去极值（1%~99%）
for col in features:
    df[col] = winsorize(df[col], limits=[0.01, 0.01])

# 标准化
df[features] = (df[features] - df[features].mean()) / df[features].std()

# 去掉未来为空的行
df = df.dropna(subset=['target'])

# 4️⃣ 划分训练集与测试集

X = df[features]

y = df['target']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)

# 5️⃣ 训练 CatBoost 模型
model = CatBoostClassifier(
    iterations=300,
    learning_rate=0.05,
    depth=6,
    loss_function='Logloss',
    verbose=0,
    random_seed=42
)
model.fit(X_train, y_train)

# ==============================
# 6️⃣ 设置字体（解决中文乱码）
# ==============================

candidate_fonts = [
    "PingFang HK",
    "PingFang SC",
    "Songti SC",
    "STHeiti",
    "Heiti TC",
    "Microsoft YaHei",
    "SimHei",
]
```

```
available_fonts = {f.name for f in font_manager.fontManager.ttflist}

for font_name in candidate_fonts:
    if font_name in available_fonts:
        break
else:
    font_name = "DejaVu Sans"

plt.rcParams["font.sans-serif"] = [font_name]
plt.rcParams["axes.unicode_minus"] = False

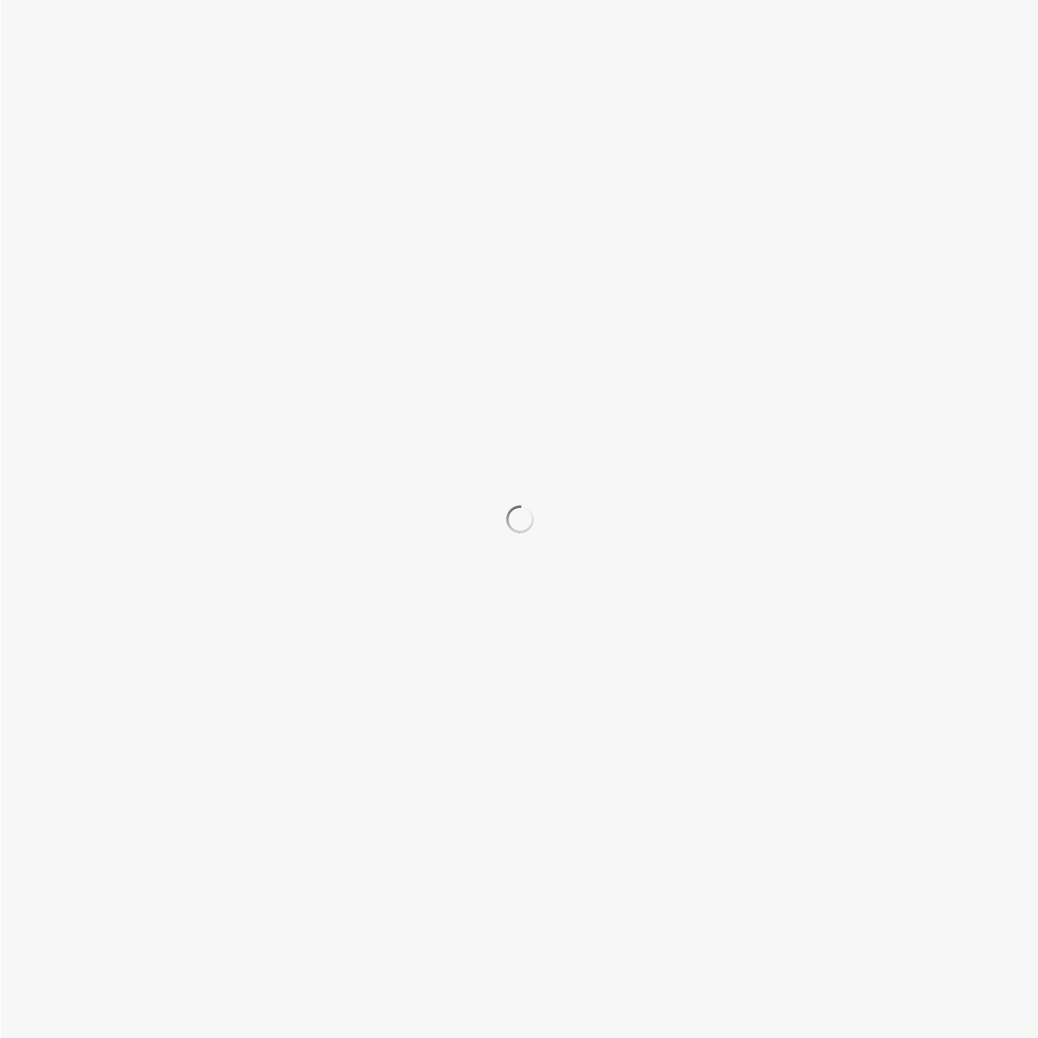
# 6️⃣ 预测与评估
y_pred_prob = model.predict_proba(X_test)[:, 1]
auc = roc_auc_score(y_test, y_pred_prob)
print(f"AUC = {auc:.4f}")

# 7️⃣ 绘制 ROC 曲线
fpr, tpr, _ = roc_curve(y_test, y_pred_prob)
plt.figure(figsize=(6,6))
plt.plot(fpr, tpr, label=f'CatBoost ROC (AUC={auc:.3f})')
plt.plot([0,1],[0,1],'--',color='gray')
plt.xlabel('False Positive Rate', fontname=font_name)
plt.ylabel('True Positive Rate', fontname=font_name)
plt.title(f'{ts_code} 未来5日涨跌预测', fontname=font_name)
plt.legend(prop={"family": font_name})
plt.grid(alpha=0.3)
plt.show()
```

5.11 运行结果解读

- 输出示例： AUC = 0.5015

AUC≈0.50 基本等于随机猜涨跌（硬币正反面），说明这版特征和模型几乎没学到有用信号；比 0.5 只高千分之一，仍可视作“没效果”。要改善，需要重新检查数据质量、扩充或派生更有预测力的因子，甚至调整标签定义或模型方案。



- 图中：
  - 曲线越靠左上角 → 模型越好
  - 对于实盘研究，可进一步加入横截面多股票训练。

5.12 优化方向建议

- 扩充/改造特征
  - 加入更多维度：换手率、成交量、波动率、市值、行业哑变量等；
  - 用指标的变化率或滚动统计（如 pe 的5日均值、pb 日变化、roe 环比、行业相对值）增强信息；
  - 引入技术面特征（MA、MACD、RSI、布林带等），丰富信号来源。
- 重定义标签与样本
  - 未来收益可换成 10 日、20 日甚至带阈值（如涨幅>1%设为1，否则0），让信号更稳定；
  - 调整时间窗口或覆盖更多年份，增加样本多样性；
  - 滑动取样或滚动训练/验证，模拟真实交易流程。
- 模型与参数
  - 尝试 LightGBM、XGBoost、随机森林等对比模型；
  - 做网格搜索或使用 CatBoost 的 GridSearch 调参；
  - 控制随机拆分方式，使用时间序列划分或交叉验证，减少过拟合。
- 数据质量检查

- 确认 roe 对齐正确，可考虑财报公告日向后平滑；
- 检查异常值、停牌数据、缺失填充是否影响结果；
- 如果样本多为噪声（A股短期波动大），可尝试做行业内排序、相对强度等预处理。

- 策略层面
  - 改为截面排序（按因子打分分层）而不是二分类；
  - 做多空组合收益回测，关注 IC/IR，而非日内方向；
  - 结合交易成本、风控条件，优化可执行性。

结论

LightGBM 与 CatBoost 形成的梯度提升框架能快速验证多因子信号、提取重要性并结合 AUC 回测评估，为收益和风险预测提供简单且可复制的起点。

量化交易入门指南 · 目录

上一篇
量化交易进阶:第48天 – 特征工程(因子标准化、缺失值填充、分箱、Winsorize 去极值)

下一篇
量化交易进阶:第50天 – 深度学习因子模型(Deep Factor Model)

阅读 111



寒冬没有退路，合理分配每份资源才能守住营地。

无尽冬日–3亿人都在玩

立即体验